Department of Computer Science and Engineering

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO1 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Duration	3 Days
Mode	Individual
Student Reg. No.	192511230
Student Name	ALURI POOJITHA

Instructions: Answer all parts. Show the basic operation, every step of each derivation and the final asymptotic result, and interpret what each result means for the platform. Assume the platform's servers execute 10^8 basic operations per second.

IT APPLICATION-BASED PROBLEM

Scenario – Scalability Audit of "ShopSphere", an Online Shopping Platform: ShopSphere is a growing e-commerce platform whose product catalogue currently holds n = 100,000 products and is expected to reach n = 1,000,000 within a year. Before the expansion, the CTO has asked you, as the platform's algorithm engineer, to audit five back-end modules, predict how each will behave as n grows, and recommend which modules must be redesigned. The modules are described below.

Module M1 – Duplicate-Listing Detector: Sellers sometimes upload the same product twice. Every night, the following routine compares every product with every later product.

```
FOR i ← 0 TO n − 2 DO
   FOR j ← i + 1 TO n − 1 DO
      IF Product[i].title = Product[j].title AND Product[i].seller =
Product[j].seller THEN
         MarkDuplicate(j)
      END IF
   END FOR
END FOR
```

Module M2 – API Rate Limiter: The public API audits request windows of doubling size using the routine below. A team member claims that it is Θ(n log n) because the outer loop runs about log n times and the inner loop runs up to n times.

```
i ← 1
WHILE i < n DO
   FOR j ← 0 TO i − 1 DO
      Audit(Request[j])          // basic operation
   END FOR
   i ← i × 2
END WHILE
```

Module M3 – Product Search: Two versions exist. Version S1 scans the unsorted catalogue sequentially until the product is found. Version S2 searches a sorted product index by

comparing the key with the middle entry and discarding half the index after each comparison; its comparison count satisfies $T(n) = T(n/2) + 1$, $T(1) = 1$.

Module M4 – Recommendation Engine: The engine splits the customer–product data into four parts of size n/2 (because of replicated partitions) and merges the partial results in linear time: $T(n) = 4T(n/2) + n$. A proposed redesign removes the replication, giving $T(n) = 2T(n/2) + n$.

Module M5 – Sales Analytics Engine: The analytics engine splits the sales data into halves recursively and sorts the records by revenue at every level, giving $T(n) = 2T(n/2) + n \log n$. Separately, profiling shows that the nightly report generator performs $f(n) = 3n^2 + 10n \log_2 n + 50$ operations.

(a) Understand the Problem: For each of the five modules, identify the input size and the basic operation that dominates the running time, explain where it occurs, and state how you expect its execution count to grow as n increases. Justify each choice briefly.

(b) Analyse the Iterative Modules: For M1, derive the exact number of comparisons as a function of n and state the time and space complexity. For M2, derive the exact number of Audit operations, decide whether the team member's $\Theta(n \log n)$ claim is correct and give the tight bound. Verify your M2 result for n = 1,024.

(c) Solve a Recurrence by Substitution: Solve the recurrence for search version S2 using backward substitution, showing every intermediate step, and derive the exact closed form. State the time complexity and the space complexity of both an iterative and a recursive implementation.

(d) Apply the Master Theorem: For M4, apply the Master Theorem to both the current design and the proposed redesign, clearly identifying a, b, f(n), n^(log_b a) and the case used, and explain what happens to the running time when the data doubles. For M5, determine whether any of the three standard cases applies to $T(n) = 2T(n/2) + n \log n$; if not, derive the complexity using the extended Master Theorem or a recursion tree.

(e) Analyse Best, Average and Worst Cases: For search versions S1 and S2, identify the input condition that produces the best, average and worst case and give the number of comparisons in each. For S1, derive the average number of comparisons when the probability that the searched product exists is p, and evaluate it for n = 1,000,000 and p = 0.5.

(f) Prove an Asymptotic Bound: Using the formal definitions, prove that the report generator's cost $f(n) = 3n^2 + 10n \log_2 n + 50$ is $\Theta(n^2)$ by finding constants c_1, c_2 and n_0, and show that f(n) is not $O(n)$.

(g) Scalability Decision: Estimate the number of basic operations and the running time of M1, M2, S2, M4 (both designs) and M5 for n = 1,000,000. Identify the modules that will fail to meet a nightly processing window of 10 minutes, and recommend, with justification, which modules must be redesigned and how.

Executive Summary

This report audits five back-end modules of ShopSphere for the planned growth from n = 100,000 to n = 1,000,000 products. Each module is analysed using asymptotic notation, recurrence solving and the Master Theorem, and the results are checked against a 10-minute nightly window on servers running 10^8 basic operations per second.

Module	**Complexity**	**Time at n = 10^6**	**Verdict**
M1 Duplicate detector	$\Theta(n^2)$	≈ 83.3 min	Redesign (hash table)
M2 Rate limiter	$\Theta(n)$	≈ 0.01 s	Keep
M3 Search S1 / S2	$\Theta(n)$ / $\Theta(\log n)$	≈ 7.5 ms (avg. at p = 0.5) / ≈ 0.2 μs per query	Use S2
M4 Recommender (current)	$\Theta(n^2)$	≈ 166.7 min	Redesign
M4 Recommender (redesign)	$\Theta(n \log n)$	≈ 0.2 s	Adopt
M5 Sales analytics	$\Theta(n \log^2 n)$	≈ 2.1 s	Keep (optional tuning)

Key finding: only the two quadratic modules, M1 and the current M4, fail the nightly window. Both can be fixed with well-understood changes.

(a) Understand the Problem

Module	**Input size**	**Basic operation and where it occurs**	**Expected growth**
M1 Duplicate detector	n = number of products	Comparison of title and seller in the IF statement of the inner loop. It is evaluated once for every pair (i, j) with i < j.	Quadratic: $\Theta(n^2)$. Ten times more products means about 100 times more comparisons.
M2 Rate limiter	n = upper bound on the window size (number of requests)	Audit(Request[j]) in the inner FOR loop. The inner loop runs i times, and i doubles on each pass of the WHILE loop.	Linear: $\Theta(n)$. The inner-loop counts form a geometric series dominated by the last term.
M3 Search (S1 and S2)	n = number of products in	S1: comparison of the key with Product[k] in the sequential	S1: linear, $\Theta(n)$. S2: logarithmic, $\Theta(\log n)$,

	the catalogue or index	scan. S2: comparison of the key with the middle entry of the current sub-range.	because the range halves each step.
M4 Recommender	n = size of the customer–product data	The linear-time merge/combine work at each recursive call (the f(n) = n term). The recursive calls multiply that work across levels.	Current design: $\Theta(n^2)$. Redesign: $\Theta(n \log n)$.
M5 Sales analytics	n = number of sales records	Comparison of revenue values in the sort performed at every level ($f(n) = n \log n$). The report generator's dominant operation is the n^2 term in $f(n) = 3n^2 + 10n \log n + 50$.	$\Theta(n \log^2 n)$ for the analytics engine. The report generator grows as $\Theta(n^2)$.

Assumptions

The audit assumes that every basic operation costs one unit of time and that the servers run 10^8 of them per second. For recurrences, n is taken to be a power of 2 and the base case costs one operation; neither choice changes the asymptotic results. Throughout, log means $\log_2$, constants and lower-order terms are dropped when a Θ bound is stated but kept in exact counts, and space is measured as the extra memory used beyond the input data.

In summary, M1 is a nested loop that compares every pair of products, so its work is governed by the number of pairs, which grows with the square of the catalogue size. M2 looks like a nested loop but its inner loop length doubles each time, so its work is governed by the last, largest window and grows only linearly. For search, S1 may have to look at every product while S2 discards half of the remaining index at each step, so the two differ between linear and logarithmic growth. M4 and M5 are divide-and-conquer modules whose cost depends on how many sub-problems are created and how much merging or sorting is done at each level. This is why the recurrences in parts (c) and (d) matter: they show how the cost accumulates across the levels of the recursion.

(b) Analyse the Iterative Modules

M1 – Duplicate-Listing Detector

Basic operation: the comparison inside the IF. There is no early exit, so it runs for every pair.

For a fixed i, j runs from i + 1 to n − 1, which is (n − 1 − i) iterations. Summing over i = 0 … n − 2:

$$C(n) = \Sigma\ (i = 0 \text{ to } n-2)\ (n - 1 - i) = (n - 1) + (n - 2) + \ldots + 2 + 1$$

$$C(n) = (n - 1)\cdot n / 2 = (n^2 - n) / 2$$

Time complexity: $C(n) = n^2/2 - n/2$, so the n^2 term dominates and the time is $\Theta(n^2)$. Best, average and worst cases are all the same.

Space complexity: $\Theta(1)$. Only the loop counters i and j are used, and the data is processed in place.

Numbers: n = 100,000 gives 4,999,950,000 ≈ 5×10^9 comparisons. n = 1,000,000 gives 499,999,500,000 ≈ 5×10^{11} comparisons.

M2 – API Rate Limiter

Basic operation: Audit(Request[j]).

The WHILE loop runs with i = 1, 2, 4, 8, …, 2^(k−1), and stops when i ≥ n. Hence the number of outer iterations is k = ⌈$\log_2$ n⌉. In the t-th pass (t = 0, 1, …, k − 1) i = 2^t, so the inner loop performs 2^t Audit operations. Total:

$$A(n) = \Sigma\ (t = 0 \text{ to } k-1)\ 2^t = 2^k - 1, \quad \text{where } k = \lceil \log_2 n \rceil$$

For n = 2^m the sum is 1 + 2 + … + 2^(m−1) = 2^m − 1 = n − 1. For a general n, 2^(k−1) < n ≤ 2^k, so $n - 1 \leq A(n) \leq 2n - 3$.

Is the Θ(n log n) claim correct? No. The claim assumes that the inner loop runs n times on every outer pass. It does not: the inner loop runs 2^t times, and these counts grow geometrically. The sum is dominated by the last term, so the total is linear. Because $n - 1 \leq A(n) \leq 2n - 3$, the tight bound is $\Theta(n)$ ($O(n)$ and $\Omega(n)$). $\Theta(n \log n)$ overestimates the work by a factor of about log n.

Verification for n = 1,024: the values of i are 1, 2, 4, 8, 16, 32, 64, 128, 256, 512 (i = 1024 stops the loop), which is 10 outer iterations. Total = 1 + 2 + 4 + … + 512 = $2^{10} - 1 = 1{,}023 = n - 1$. The team member's formula would predict $n \log_2 n = 1{,}024 \times 10 = 10{,}240$, about 10 times too large.

Space complexity: $\Theta(1)$.

What the M1 and M2 results mean for ShopSphere

Module	**n = 100,000**	**n = 1,000,000**	**Growth when n × 10**
M1: n(n−1)/2	≈ 5×10^9 (≈ 50 s)	≈ 5×10^{11} (≈ 83 min)	× 100

M2: 2^⌈log n⌉ − 1	131,071 (≈ 1.3 ms)	1,048,575 (≈ 10.5 ms)	× 8 (≈ × 10 in the limit)

M1 is cheap today but explodes with growth: a tenfold increase in catalogue size means a hundredfold increase in nightly work. M2 stays negligible, so its design is sound and no action is needed. The M2 figures for n = 100,000 use $k = \lceil \log_2 100{,}000 \rceil = 17$, giving $2^{17} - 1 = 131{,}071$.

(c) Solve a Recurrence by Substitution (Search S2)

Recurrence: T(n) = T(n/2) + 1, T(1) = 1. Assume n = 2^k. Backward substitution:

T(n) = T(n/2) + 1

= [T(n/4) + 1] + 1 = T(n/4) + 2

= [T(n/8) + 1] + 2 = T(n/8) + 3

…

= T(n/2^k) + k (after k substitutions)

The recursion stops when n/2^k = 1, i.e. $k = \log_2 n$. Then T(n/2^k) = T(1) = 1, so:

$T(n) = 1 + \log_2 n$ (exact closed form for n = 2^k)

For general n the closed form is $T(n) = \lfloor \log_2 n \rfloor + 1$. Check: T(1) = 1, T(2) = 2, T(4) = 3, T(8) = 4, which agrees with $1 + \log_2 n$. For n = 1,000,000 this is $\lfloor 19.93 \rfloor + 1 = 20$ comparisons.

Implementation	**Time complexity**	**Space complexity**	**Reason**
Iterative binary search	Θ(log n)	Θ(1)	Only low, high and mid are stored; no call stack is used.
Recursive binary search	Θ(log n)	Θ(log n)	The recursion depth is about $\log_2$ n, and each call needs a stack frame.

What the S2 result means for ShopSphere

Catalogue size n	**S1 worst case (n)**	**S2 worst case (⌊log n⌋ + 1)**
1,024	1,024	11
100,000	100,000	17
1,048,576 (2^{20})	1,048,576	21

Growing the catalogue ten-fold adds only about 3 or 4 comparisons to S2 (each doubling adds exactly one), while S1 does ten times more work. A sorted index therefore scales almost for free. The cost is

keeping the index sorted when products are added, which is a maintenance cost rather than a per-query cost.

The two implementations of binary search therefore take the same time but differ in memory. The iterative version keeps only three indices and so uses constant space, while the recursive version adds one stack frame per halving and so needs space proportional to log n. For a catalogue of a million products that is only about 20 frames, so either version is acceptable on ShopSphere's servers, but the iterative one is the safer choice in a service that handles very many concurrent searches.

(d) Apply the Master Theorem

Master Theorem: for T(n) = aT(n/b) + f(n) with a ≥ 1 and b > 1, compare f(n) with n^(log_b a). Case 1: f(n) = O(n^(log_b a − ε)) gives Θ(n^(log_b a)). Case 2: f(n) = Θ(n^(log_b a)) gives Θ(n^(log_b a) log n). Case 3: f(n) = Ω(n^(log_b a + ε)) with the regularity condition gives Θ(f(n)).

	M4 current: T(n) = 4T(n/2) + n	**M4 redesign: T(n) = 2T(n/2) + n**
a, b	a = 4, b = 2	a = 2, b = 2
f(n)	n	n
n^(log_b a)	$n^{(\log_2 4)} = n^2$	$n^{(\log_2 2)} = n$
Comparison	f(n) = n = O(n^(2 − ε)) with ε = 1	f(n) = n = Θ(n^(log_b a))
Case	Case 1	Case 2
Result	$T(n) = \Theta(n^2)$	T(n) = Θ(n log n)

What happens when the data doubles: Current design: $T(2n)/T(n) = (2n)^2/n^2 = 4$, so the running time quadruples. Redesign: $T(2n)/T(n) = 2n \cdot \log(2n) / (n \cdot \log n) = 2(1 + 1/\log n)$, which is slightly more than 2 (about 2.10 for $n = 10^6$). Removing the replication changes the growth from quadratic to near-linear, so the cost of expansion becomes manageable.

M5 – T(n) = 2T(n/2) + n log n

Here a = 2, b = 2 and n^(log_b a) = n, while f(n) = n log n.

Case 1 fails: f(n) = n log n is larger than n, so it is not O(n^(1 − ε)). **Case 2 (basic form) fails:** f(n) is not Θ(n), because the extra log n factor is unbounded. **Case 3 fails:** f(n) would have to be Ω(n^(1 + ε)) for some ε > 0, but log n = o(n^ε) for every ε > 0, so n log n is not polynomially larger than n.

None of the three standard cases applies; the recurrence falls in the gap between Case 2 and Case 3.

Extended Master Theorem: if f(n) = Θ(n^(log_b a) · log^k n) with k ≥ 0, then T(n) = Θ(n^(log_b a) · log^(k+1) n). Here k = 1, so $T(n) = \Theta(n \log^2 n)$.

Recursion-tree verification (n = 2^m, T(1) = 1): At level i there are 2^i subproblems of size n/2^i,

each costing $(n/2^i)\cdot\log(n/2^i) = (n/2^i)(m - i)$. The cost of level i is therefore $n(m - i)$. Summing levels $0 \ldots m - 1$ and adding the n leaves:

$$T(n) = n \cdot \Sigma\,(i = 0 \text{ to } m-1)\,(m - i) + n = n \cdot m(m + 1)/2 + n$$

$$= (n/2)(\log^2 n + \log n) + n = \Theta(n \log^2 n)$$

Recursion-tree check for M4 (current design)

For $T(n) = 4T(n/2) + n$ with $n = 2^m$: level i has 4^i subproblems of size $n/2^i$, each costing $n/2^i$, so the level cost is $4^i \cdot n/2^i = n \cdot 2^i$. The costs grow geometrically, so the leaves dominate. Summing levels $0 \ldots m - 1$ gives $n(2^m - 1) = n(n - 1)$, and the n leaves add $n \cdot T(1) = n$, so $T(n) = n^2 = \Theta(n^2)$. This agrees with Case 1.

For the redesign, level i has 2^i subproblems of size $n/2^i$, so each level costs n. With log n levels plus n leaves, $T(n) = n \log n + n = \Theta(n \log n)$, which agrees with Case 2.

What the M4 and M5 results mean for ShopSphere

Design	**n = 100,000**	**n = 1,000,000**	**Growth when n × 10**
M4 current: n^2	10^{10} (≈ 100 s)	10^{12} (≈ 2.8 h)	× 100
M4 redesign: n log n	≈ 1.66×10^6 (≈ 0.017 s)	≈ 2.0×10^7 (≈ 0.2 s)	≈ × 12
M5: $n \log^2 n$ (order)	≈ 2.8×10^7 (≈ 0.28 s)	≈ 4×10^8 (≈ 4 s)	≈ × 14

The replicated partitions in the current M4 are what push the recurrence into the n^2 regime: four sub-problems of half size means the work does not shrink as the recursion deepens. The M5 figures use the order $n \log^2 n$ (the exact count is about half of that, as derived above), so they are conservative.

To explain the two Master Theorem results in words: in the current M4 design the four sub-problems of half size generate so much work at the bottom of the recursion that the leaves dominate and the total is quadratic. After the redesign only two half-size sub-problems remain, so every level of the recursion costs about the same amount, n, and the total is n multiplied by the number of levels, log n. For M5 the sorting done at each level costs slightly more than the linear merge in M4's redesign, which is why the Master Theorem's three standard cases cannot decide it and the extended version is needed; the result, $n \log^2 n$, is only one log n factor above the redesigned M4.

(e) Best, Average and Worst Cases

Version	**Best case**	**Average case**	**Worst case**
S1 – sequential search	Key is the first product: 1 comparison, $\Theta(1)$	Key at a random position: $(n + 1)/2$ comparisons for a successful search, $\Theta(n)$	Key is the last product or absent: n comparisons, $\Theta(n)$
S2 – binary search	Key equals the middle entry: 1 comparison, $\Theta(1)$	Key found at a random position, or absent: about $\log_2 n - 1$ (successful) to $\log_2 n$ (unsuccessful) comparisons, $\Theta(\log n)$	Key found at the last level, or absent: $\lfloor \log_2 n \rfloor + 1$ comparisons, $\Theta(\log n)$

Average case of S1 with probability p that the product exists

If the product exists (probability p), it is equally likely to be at any of the n positions, each with probability p/n. Finding it at position i costs i comparisons. If it does not exist (probability $1 - p$), all n products are compared.

$$C_avg(n) = \Sigma\ (i = 1 \text{ to } n)\ i \cdot (p/n) + n(1 - p)$$

$$= (p/n) \cdot n(n + 1)/2 + n(1 - p)$$

$$= p(n + 1)/2 + n(1 - p)$$

Checks: $p = 1$ gives $(n + 1)/2$ and $p = 0$ gives n. For $n = 1{,}000{,}000$ and $p = 0.5$:

$C_avg = 0.5 \times 1{,}000{,}001/2 + 0.5 \times 1{,}000{,}000 = 250{,}000.25 + 500{,}000 = 750{,}000.25 \approx 7.5 \times 10^5$ comparisons

Compare this with at most 20 comparisons for S2 on the same catalogue.

Growth of S1's average cost with p

p (product exists)	**$C_avg = p(n+1)/2 + n(1 - p)$ at $n = 10^6$**	**Interpretation**
1.0	500,000.5	Always found, about half the catalogue scanned
0.5	750,000.25	Mixed workload
0.25	875,000.125	Mostly misses, close to the worst case
0.0	1,000,000	Always a miss, the full catalogue is scanned

The average cost rises as the hit rate falls, because an unsuccessful search must examine every product. Either way S1 stays linear, so the $\Theta(n)$ behaviour does not depend on p.

In words, the results above show that the best case for either search is a lucky first probe, while the worst case separates the two sharply: a sequential scan may examine all one million products, whereas binary search never needs more than 20. The average case for S1 depends on how often searches succeed. When searches always succeed the scan covers about half the catalogue on average, and when they never succeed it covers all of it. With p = 0.5, as in the question, the result is about 750,000 comparisons, which is 75% of the worst case.

(f) Prove an Asymptotic Bound

Definition: $f(n) = \Theta(g(n))$ if there exist constants $c_1, c_2 > 0$ and n_0 such that $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$. Here $f(n) = 3n^2 + 10n \log_2 n + 50$ and $g(n) = n^2$.

Lower bound: since $10n \log_2 n \geq 0$ and $50 > 0$ for $n \geq 1$, $f(n) \geq 3n^2$. So $c_1 = 3$ works for all $n \geq 1$.

Upper bound: for $n \geq 1$ we have $\log_2 n \leq n$, hence $n \log_2 n \leq n^2$, and also $50 \leq 50n^2$. Therefore:

$$f(n) \leq 3n^2 + 10n^2 + 50n^2 = 63n^2 \quad \text{for all } n \geq 1$$

So $c_2 = 63$ works.

Constants: $c_1 = 3$, $c_2 = 63$, $n_0 = 1$. Then $3n^2 \leq f(n) \leq 63n^2$ for all $n \geq 1$, hence $f(n) = \Theta(n^2)$. (Check at n = 1: $f(1) = 53$, and $3 \leq 53 \leq 63$.)

f(n) is not O(n): Suppose $f(n) \leq c \cdot n$ for some constant $c > 0$ and all $n \geq n_0$. Since $f(n) \geq 3n^2$, we would have $3n^2 \leq c \cdot n$, i.e. $n \leq c/3$ for all $n \geq n_0$. This fails for any $n > \max(n_0, c/3)$. This contradiction shows that no such c and n_0 exist, so $f(n) \notin O(n)$.

Numerical sanity check of the $\Theta(n^2)$ bound

n	**$f(n) = 3n^2 + 10n \log_2 n + 50$**	**$3n^2$ (lower)**	**$63n^2$ (upper)**	**$f(n)/n^2$**
10	≈ 682.2	300	6,300	6.82
100	≈ 36,693.9	30,000	630,000	3.67
1,000	≈ 3,099,708	3,000,000	63,000,000	3.10
1,000,000	$\approx 3.0002 \times 10^{12}$	3×10^{12}	6.3×10^{13}	3.0002

The ratio $f(n)/n^2$ approaches 3 as n grows, which is why the n^2 term decides the growth rate and the other terms become irrelevant. A tighter upper constant is also possible: $c_2 = 4$ works for $n \geq 64$, because $10n \log_2 n + 50 \leq n^2$ from n = 64 onwards (at n = 64: 3,890 ≤ 4,096).

The proof works by squeezing f(n) between two multiples of n^2. The lower bound comes directly from the leading term, since the other two terms are never negative. The upper bound comes from observing that, for $n \geq 1$, both $n \log_2 n$ and the constant 50 are at most a constant multiple of n^2. Showing that f(n) is not O(n) uses a proof by contradiction: if a linear bound existed, then $3n^2 \leq c \cdot n$ would have to hold for all large n, which is impossible because n^2 eventually exceeds any constant multiple of n. Together these facts show that the report generator grows exactly like n^2, so it is the n^2 term, and not the n log n term or the constant, that determines its scalability.

(g) Scalability Decision for n = 1,000,000

Window: 10 minutes = 600 s, which allows $600 \times 10^8 = 6 \times 10^{10}$ operations. Time = operations / 10^8.

Module	**Operations at $n = 10^6$**	**Running time**	**Fits 10-min window?**
M1 (n(n−1)/2)	$\approx 5 \times 10^{11}$	≈ 5,000 s ≈ 83.3 min	No
M2 (2^⌈log n⌉ − 1)	1,048,575 ≈ 1.05×10^6	≈ 0.0105 s	Yes
S2 (⌊log n⌋ + 1) per search	20	$\approx 2 \times 10^{-7}$ s	Yes
M4 current (n^2)	$\approx 10^{12}$	≈ 10,000 s ≈ 166.7 min (2.8 h)	No
M4 redesign (n log n)	$\approx 2.0 \times 10^7$	≈ 0.2 s	Yes
M5 (n·L(L+1)/2 + n, L = log n)	$\approx 2.1 \times 10^8$ (order $n \log^2 n \approx 4 \times 10^8$)	≈ 2.1 s	Yes

Modules that fail the window

M1 (about 83 minutes) and the current M4 design (about 2.8 hours) exceed the 10-minute window. Both are quadratic, so they were tolerable at n = 100,000 (M1 ≈ 50 s, M4 ≈ 100 s) but become unacceptable at ten times the data, since time grows 100-fold.

Recommendations

Redesign M1 (mandatory). Replace the all-pairs comparison. Insert each product's (title, seller) key into a hash table and flag a product whose key already exists. This takes Θ(n) expected time (about 10^6 operations, ≈ 0.01 s) and Θ(n) extra space. Alternatively, sort by (seller, title) in Θ(n log n) and compare adjacent records (≈ 0.2 s). Trading Θ(n) space for the drop from n^2 to n is clearly worthwhile.

Redesign M4 (mandatory). Adopt the proposed redesign without replicated partitions, i.e. $T(n) = 2T(n/2) + n = \Theta(n \log n)$. This cuts the time from about 10^4 s to about 0.2 s, and doubling the data then costs only slightly over 2× instead of 4×.

Keep M2, S2 and M5. M2 is linear and negligible. S2 needs only about 20 comparisons per query, so keep the sorted index and retire S1, whose average of 7.5×10^5 comparisons per query (at $p = 0.5$) does not scale with traffic. M5 finishes in about 2 s; it already fits, though sorting once and merging ($T(n) = 2T(n/2) + n = \Theta(n \log n)$) would remove the extra log n factor.

Additional finding. The nightly report generator is $\Theta(n^2)$: $f(10^6) \approx 3 \times 10^{12}$ operations ≈ 30,000 s ≈ 8.3 hours, which would also miss the window. It should be reworked (for example by aggregating in a single pass or in $\Theta(n \log n)$) even though it was not listed in this part.

Reading the table above, M1 and the current M4 are the only modules whose estimated time exceeds the 600-second window, and by a wide margin: M1 needs about eight times the allowed time and the current M4 about seventeen times. The remaining modules finish in a few seconds at most, so they pose no scalability risk at one million products. The decision is therefore not about tuning constants but about changing the algorithms for M1 and M4, because no hardware speed-up of a realistic size would bring a quadratic module back inside the window if the catalogue keeps growing.

Before and after the redesigns (n = 1,000,000)

Module	**Before**	**After**	**Speed-up**
M1	$\Theta(n^2)$: ≈ 5,000 s	Hash set $\Theta(n)$: ≈ 0.01 s	≈ 500,000 ×
M4	$\Theta(n^2)$: ≈ 10,000 s	$\Theta(n \log n)$: ≈ 0.2 s	≈ 50,000 ×
Search	S1 avg. ≈ 7.5×10^5 comparisons	S2 ≈ 20 comparisons	≈ 37,500 ×

Pseudocode of the recommended M1 redesign

```
seen ← empty hash set
FOR i ← 0 TO n − 1 DO
   key ← (Product[i].title, Product[i].seller)
   IF key IS IN seen THEN
```

```
        MarkDuplicate(i)
    ELSE
        INSERT key INTO seen
    END IF
END FOR
```

Each product is processed once, with an expected constant-time hash lookup, so the time is $\Theta(n)$ expected. The hash set needs $\Theta(n)$ extra space. The output is the same as the original routine: the later copy of each pair is marked as the duplicate.

Order of growth of the functions met in this audit

$$1 < \log n < n < n \log n < n \log^2 n < n^2$$

S2 sits at log n, M2 and the hash-based M1 at n, the M4 redesign at n log n, M5 at $n \log^2 n$, and the original M1, the current M4 and the report generator at n^2. Anything at n^2 or worse is the risk at ten times the data.

Conclusion

The audit shows that scalability is decided by the order of growth, not by today's running time. Modules that look fast at 100,000 products (M1, M4 current, the report generator) become the bottleneck at 1,000,000 because their cost is quadratic. The recommended order of work is: first M1, then M4 (adopt the redesign), then the report generator, and finally replace S1 with S2 wherever it is still used. With these changes every module runs well inside the 10-minute window, with a large safety margin for further growth.

Rubrics – Application-Based Question

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Criteria	**Weightage (%)**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Code of Conduct

I Aluri poojitha(192511230)_____ (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Poojitha_____ Date: 8-10-26_____